

01 | Count positive values – Problem

EASY Counter + condition

[Try it first](#)

Your task. Return how many elements of an integer array are strictly positive. Zero does not count.

$0 \leq n \leq 1000$; values between -1000 and 1000 . Leave the input unchanged.

Examples

`[-2, 0, 5, 3, -1] → 2`

`[0, -4] → 0`

Implement this function

```
int countPositive(const std::vector<int>& nums);
```

`#include <vector>`. A vector is a resizable array: `nums[i]` accesses an element; `nums.size()` gives its length.

Think first: What single value should you keep while reading the array?

01 | Count positive values – Solution

```
int countPositive(const std::vector<int>& nums)
{
    int count = 0;
    int n = static_cast<int>(nums.size());
    for (int i = 0; i < n; ++i)
    {
        if (nums[i] > 0)
        {
            ++count;
        }
    }
    return count;
}
```

The idea

Start a counter at zero. Increase it only when the current element passes the condition.

Trace it

For $[-2, 0, 5, 3, -1]$, the counter becomes 0, 0, 1, 2, 2.

Check the edge

An empty array returns 0: the loop never runs.

$O(n)$ time; $O(1)$ extra space.

02 | Running sum – Problem

EASY Accumulate a result

[Try it first](#)

Your task. Return a new array whose element at index i is the sum of the input elements from index 0 through i .

$0 \leq n \leq 1000$; values between -1000 and 1000 . Leave the input unchanged.

Examples

$[2, 1, 3, 4] \rightarrow [2, 3, 6, 10]$

$[-1, 2, -3] \rightarrow [-1, 1, -2]$

Implement this function

```
std::vector<int> runningSum(  
    const std::vector<int>& nums);
```

`#include <vector>`. `std::vector<int> result(n)` creates n zero-initialized integer elements.

Think first: How can you reuse the sum from the previous iteration?

02 | Running sum – Solution

```
std::vector<int> runningSum(  
    const std::vector<int>& nums)  
{  
    int n = static_cast<int>(nums.size());  
    std::vector<int> result(n);  
    int total = 0;  
    for (int i = 0; i < n; ++i)  
    {  
        total += nums[i];  
        result[i] = total;  
    }  
    return result;  
}
```

The idea

Keep a running total. Add the current element, then store that total in the output.

Trace it

For $[2, 1, 3, 4]$, the total evolves as $2 \rightarrow 3 \rightarrow 6 \rightarrow 10$.

Check the edge

An empty input produces an empty output. Negative values also work.

$O(n)$ time; $O(n)$ output space and $O(1)$ other space.

03 | Richest customer – Problem

EASY Nested loops + maximum

[Try it first](#)

Your task. Each row of accounts contains one customer's balances at several banks. Return the largest row total.

$1 \leq m, b \leq 100$; m rows, b entries per row; balances from 0 to 1000.

Examples

`[[2, 3], [1, 8], [4, 4]]` $\rightarrow$ 9

`[[0, 0]]` $\rightarrow$ 0

Implement this function

```
int maximumWealth(  
    const std::vector<std::vector<int>>& accounts);
```

#include <vector> and <algorithm>. `accounts[r][c]` is row r , column c ; `std::max(a,b)` returns the larger value.

Think first: Which variable must reset when you move to the next customer?

03 | Richest customer – Solution

```
int maximumWealth(  
    const std::vector<std::vector<int>>& accounts)  
{  
    int best = 0;  
    int rows = static_cast<int>(accounts.size());  
    for (int r = 0; r < rows; ++r)  
    {  
        int total = 0;  
        int b = static_cast<int>(accounts[r].size());  
        for (int c = 0; c < b; ++c)  
        {  
            total += accounts[r][c];  
        }  
        best = std::max(best, total);  
    }  
    return best;  
}
```

The idea

The inner loop sums one row. The outer loop compares that total with the best total so far.

Trace it

Row totals are 5, 9, 8. The best total becomes $5 \rightarrow 9 \rightarrow 9$.

Check the edge

Reset total inside the outer loop. All-zero balances correctly give zero.

$O(mb)$ time; $O(1)$ extra space.

04 | Missing number – Problem

EASY Use a known total

[Try it first](#)

Your task. An array of length n contains distinct integers from 0 through n , with exactly one number missing. Return it.

$1 \leq n \leq 1000$; no duplicates; the input may be unsorted.

Examples

`[3, 0, 1] → 2`

`[0, 1] → 2`

Implement this function

```
int missingNumber(const std::vector<int>& nums);
```

`#include <vector>`. There are $n + 1$ possible values, but only n array elements.

Think first: Compare the actual sum with $0 + 1 + \dots + n = n(n + 1)/2$.

04 | Missing number – Solution

```
int missingNumber(const std::vector<int>& nums)
{
    int n = static_cast<int>(nums.size());
    int expected = n * (n + 1) / 2;
    int actual = 0;
    for (int i = 0; i < n; ++i)
    {
        actual += nums[i];
    }
    return expected - actual;
}
```

The idea

Every present number appears in both totals. Subtraction leaves exactly the missing number.

Trace it

For $[3, 0, 1]$: $\text{expected} = 6$, $\text{actual} = 4$; the answer is 2.

Check the edge

The missing value can be 0 or n . The stated bound keeps int arithmetic safe.

$O(n)$ time; $O(1)$ extra space.

05 | Reverse a string – Problem

EASY Two indices + swapping

[Try it first](#)

Your task. Reverse the characters of a string in place. Modify the input instead of returning a second string.

$0 \leq n \leq 1000$; lowercase English letters only. Use $O(1)$ extra space.

Examples

"hello" → "olleh"

"ab" → "ba"

Implement this function

```
void reverseString(std::string& s);
```

#include <string>. s[i] accesses a character; std::string& lets the function change the caller's string.

Think first: Which two characters should exchange positions first?

05 | Reverse a string – Solution

```
void reverseString(std::string& s)
{
    int left = 0;
    int right = static_cast<int>(s.size()) - 1;
    while (left < right)
    {
        char temp = s[left];
        s[left] = s[right];
        s[right] = temp;
        ++left;
        --right;
    }
}
```

The idea

Swap the two outer characters, then move both indices inward. Stop when they meet or cross.

Trace it

hello → oellh → olleh. The middle character stays in place.

Check the edge

For an empty string, `right = -1`; the loop is skipped. Convert the size before subtracting.

$O(n)$ time; $O(1)$ extra space.

06 | Is it a palindrome? – Problem

EASY Two indices + early return

[Try it first](#)

Your task. Return true if a string reads the same forward and backward. Compare the characters exactly as given.

$0 \leq n \leq 1000$; lowercase English letters only. An empty string is a palindrome.

Examples

"level" → true

"hello" → false

Implement this function

```
bool isPalindrome(const std::string& s);
```

`#include <string>`. `const std::string&` reads the existing string without copying or modifying it.

Think first: What observation lets you stop before checking every character?

06 | Is it a palindrome? – Solution

```
bool isPalindrome(const std::string& s)
{
    int left = 0;
    int right = static_cast<int>(s.size()) - 1;
    while (left < right)
    {
        if (s[left] != s[right])
        {
            return false;
        }
        ++left;
        --right;
    }
    return true;
}
```

The idea

Compare matching positions from the outside inward. One mismatch proves the answer is false.

Trace it

level: compare l/l, then e/e.
No mismatch, so return true.

Check the edge

Return true after the loop, once every required pair has passed. A single character also passes.

$O(n)$ worst-case time; $O(1)$ extra space.

07 | Move zeroes – Problem

EASY Read index + write index

[Try it first](#)

Your task. Move all zeroes to the end of the array in place. Preserve the relative order of the nonzero elements.

$0 \leq n \leq 1000$; values between -1000 and 1000 . Use $O(1)$ extra space.

Examples

`[0, 4, 0, 1, 3] → [4, 1, 3, 0, 0]`

`[0, 0] → [0, 0]`

Implement this function

```
void moveZeroes(std::vector<int>& nums);
```

`#include <vector>`. `std::vector<int>&` allows changes to the original array. Its length must stay the same.

Think first: First collect nonzero values at the front. What should fill the remaining positions?

07 | Move zeroes – Solution

```
void moveZeroes(std::vector<int>& nums)
{
    int n = static_cast<int>(nums.size());
    int write = 0;
    for (int read = 0; read < n; ++read)
    {
        if (nums[read] != 0)
        {
            nums[write] = nums[read];
            ++write;
        }
    }
    for (int i = write; i < n; ++i)
    {
        nums[i] = 0;
    }
}
```

The idea

Copy each nonzero value to the next free position at the front, then fill the unused tail with zeroes.

Trace it

The prefix becomes [4,1,3]; write = 3. Fill indices 3 and 4 with zeroes.

Check the edge

write never passes read, so no unread value is overwritten.

All-zero input works.

$O(n)$ time for two passes; $O(1)$ extra space.

08 | Two Sum – Problem

EASY Try every distinct pair

Try it first

Your task. Return the two **indices** of elements whose sum equals target. You must use two different positions.

$2 \leq n \leq 1000$; values and target between -10^4 and 10^4 . Exactly one valid pair exists; either index order is accepted.

Examples

```
nums = [4, 7, 1, 9], target = 10 → [2, 3]
```

```
nums = [3, 3], target = 6 → [0, 1]
```

Implement this function

```
std::vector<int> twoSum(
```

```
    const std::vector<int>& nums, int target);
```

```
#include <vector>. return {i, j}; constructs a two-element result vector. Indices start at zero.
```

Think first: For a fixed index i , start the second index at $i + 1$.

08 | Two Sum – Solution

```
std::vector<int> twoSum(  
    const std::vector<int>& nums, int target)  
{  
    int n = static_cast<int>(nums.size());  
    for (int i = 0; i < n; ++i)  
    {  
        for (int j = i + 1; j < n; ++j)  
        {  
            if (nums[i] + nums[j] == target)  
            {  
                return {i, j};  
            }  
        }  
    }  
    return {}; // Defensive fallback  
}
```

The idea

Test each pair once. Starting at $i + 1$ prevents reusing one element and avoids checking reversed pairs.

Trace it

Values $1 + 9 = 10$ are at indices 2 and 3. Return $[2, 3]$.

Check the edge

Equal values at different indices are allowed. The guarantee makes the fallback unnecessary.

$O(n^2)$ time; $O(1)$ extra space.
Suitable for these small inputs.

09 | Best time to buy and sell – Problem

EASY Track the best value so far

[Try it first](#)

Your task. Given daily prices, return the maximum profit from at most one buy followed by one sell on a later day. You may choose no trade.

$1 \leq n \leq 1000$; prices from 0 to 10^4 . Return 0 if no positive profit is possible.

Examples

[7, 1, 5, 3, 6, 4] → 5 (buy at 1, sell at 6)

[7, 6, 4, 3, 1] → 0

Implement this function

```
int maxProfit(const std::vector<int>& prices);
```

#include <vector> and <algorithm>. std::min and std::max select the smaller and larger values.

Think first: If you sell today, which price from an earlier day would you want to have paid?

09 | Best time to buy and sell – Solution

```
int maxProfit(const std::vector<int>& prices)
{
    int n = static_cast<int>(prices.size());
    int minPrice = prices[0];
    int best = 0;
    for (int i = 1; i < n; ++i)
    {
        int profit = prices[i] - minPrice;
        best = std::max(best, profit);
        minPrice = std::min(minPrice, prices[i]);
    }
    return best;
}
```

The idea

Before each iteration, minPrice is the cheapest earlier price. Evaluate selling today, then update that minimum.

Trace it

At price 6, the earlier minimum is 1, so the candidate profit is $6 - 1 = 5$.

Check the edge

One day or decreasing prices gives 0. Choosing the global minimum and maximum can reverse time.

$O(n)$ time; $O(1)$ extra space.

10 | Binary search – Problem

EASY Halve a sorted search interval

Try it first

Your task. Given an array sorted in strictly increasing order, return the index of `target`, or `-1` if it is absent.

$0 \leq n \leq 1000$; integer values; no duplicates. Aim for $O(\log n)$ time when $n > 1$.

Examples

```
nums = [-3, 0, 4, 8, 12], target = 8 → 3
```

```
nums = [-3, 0, 4, 8, 12], target = 5 → -1
```

Implement this function

```
int search(const std::vector<int>& nums, int target);
```

`#include <vector>`. Keep an inclusive interval `[left, right]`; only those positions may still contain the target.

Think first: After inspecting the middle value, which half can you rule out?

10 | Binary search – Solution

```
int search(const std::vector<int>& nums, int target)
{
    int left = 0;
    int right = static_cast<int>(nums.size()) - 1;
    while (left <= right)
    {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) { return mid; }
        if (nums[mid] < target) { left = mid + 1; }
        else { right = mid - 1; }
    }
    return -1;
}
```

The idea

Sorted order rules out half the remaining positions. Exclude mid so the interval always shrinks.

Trace it

Target 8: index 2 holds 4. Keep [3, 4]; index 3 holds 8.

Check the edge

$\text{left} \leq \text{right}$ includes the last candidate. Empty input returns -1.

$O(\log n)$ time for $n > 1$; $O(1)$ extra space.